

UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
Escuela Profesional de Ingeniería de Sistemas



**Proyecto “Sistema web de detección y análisis de anomalías en
operaciones de bases de datos NoSQL, 2026”**
Documento de Arquitectura de Software (SAD)

Curso: BASE DE DATOS 2
Docente: PATRICK JOSE CUADROS QUIROGA

Integrantes:
MANGO PUMA, Franco Teddy
CALDERON RODRIGUEZ, Juan Carlos

Tacna - Perú
2026



Sistema web de detección y análisis de anomalías en operaciones de bases de datos NoSQL, 2026

Documento de Arquitectura de Software

Versión 1.0

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	FM / JC	FM / JC	Pendiente de validación académica	26/09/2026	Elaboración del SAD conforme al formato EPIS, alineado con el SRS y la implementación del repositorio.

Nota. La aprobación académica deberá registrarse únicamente cuando exista conformidad formal. El documento describe el estado de la arquitectura del MVP y distingue los controles requeridos para un despliegue externo.



ÍNDICE GENERAL

CONTROL DE VERSIONES.....	2
ÍNDICE GENERAL.....	3
1. INTRODUCCIÓN.....	4
1.1. Propósito (Diagrama 4+1).....	4
1.2. Alcance.....	5
1.3. Definición, siglas y abreviaturas.....	5
1.4. Organización del documento.....	6
2. OBJETIVOS Y RESTRICCIONES ARQUITECTÓNICAS.....	7
2.1. Priorización de requerimientos.....	7
2.1.1. Requerimientos Funcionales.....	7
2.1.2. Requerimientos No Funcionales - Atributos de Calidad.....	8
2.2. Restricciones.....	10
3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA.....	12
3.1. Vista de Caso de uso.....	12
3.1.1. Diagramas de Casos de uso.....	13
3.2. Vista Lógica.....	14
3.2.1. Diagrama de Subsistemas (paquetes).....	14
3.2.2. Diagrama de Secuencia (vista de diseño).....	16
3.2.3. Diagrama de Colaboración (vista de diseño).....	18
3.2.4. Diagrama de Objetos.....	19
3.2.5. Diagrama de Clases.....	20
3.2.6. Diagrama de Base de datos (NoSQL).....	21
3.3. Vista de Implementación (vista de desarrollo).....	23
3.3.1. Diagrama de arquitectura software (paquetes).....	23
3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes).....	24
3.4. Vista de procesos.....	26
3.4.1. Diagrama de Procesos del sistema (diagrama de actividad).....	26
3.5. Vista de Despliegue (vista física).....	27
3.5.1. Diagrama de despliegue.....	27
4. ATRIBUTOS DE CALIDAD DEL SOFTWARE.....	29
4.1. Escenario de Funcionalidad.....	29
4.2. Escenario de Usabilidad.....	29
4.3. Escenario de Confiabilidad.....	30
4.4. Escenario de Rendimiento.....	30
4.5. Escenario de Mantenibilidad.....	31
4.6. Otros escenarios.....	31
Seguridad.....	31
Portabilidad y despliegue.....	32
Observaciones para el despliegue próximo.....	33



1. INTRODUCCIÓN

El presente Documento de Arquitectura de Software (SAD) define la organización estructural y las decisiones técnicas del Sistema web de detección y análisis de anomalías en operaciones de bases de datos NoSQL. El documento se apoya en el Documento de Visión, el Informe de Factibilidad, el SRS y la implementación del MVP disponible en el repositorio del proyecto.

La solución se implementa como una aplicación web cliente-servidor con backend FastAPI organizado en módulos, frontend HTML/CSS/JavaScript servido por la misma aplicación y persistencia MongoDB. El sistema recibe eventos, valida su estructura, los almacena, ejecuta reglas de detección, calcula puntuación y severidad y, cuando corresponde, crea o agrupa una alerta para revisión humana.

1.1. Propósito (Diagrama 4+1)

El propósito del SAD es proporcionar una visión común de la arquitectura a partir del modelo 4+1. La vista de casos de uso representa los escenarios funcionales que condicionan el diseño; la vista lógica describe subsistemas, entidades y relaciones; la vista de desarrollo muestra la organización del código y componentes; la vista de procesos explica la ejecución principal; y la vista física describe el despliegue. Estas vistas se relacionan con los requisitos funcionales y no funcionales definidos en el SRS.

Vista 4+1	Objetivo en el proyecto	Elementos principales
Casos de uso	Validar la arquitectura mediante los flujos de mayor cobertura funcional.	Autenticación, eventos, alertas, dashboard, análisis y gestión de analistas.
Lógica	Representar responsabilidades, módulos, entidades y dependencias.	Frontend, rutas, auth, schemas, detección, servicios, acceso a datos y MongoDB.
Desarrollo / implementación	Mostrar cómo la vista lógica se materializa en el repositorio.	backend/app, frontend, simulator, scripts y requirements.txt.
Procesos	Explicar el orden de ejecución y colaboración durante el procesamiento.	Validación, persistencia, detección, alertamiento y consulta.
Despliegue	Mostrar los nodos físicos y comunicaciones previstas.	Navegador, servidor FastAPI, fuente de eventos y MongoDB.

1.2. Alcance

El SAD cubre la arquitectura del MVP actual y el escenario de despliegue de bajo volumen definido para evaluación académica. Incluye los componentes que ya forman parte del repositorio y señala las medidas necesarias antes de exponer el sistema en un entorno externo.

- Frontend web con login, dashboard, eventos, alertas, estadísticas, análisis de actividad y administración de usuarios.
- API REST en FastAPI con rutas para autenticación, usuarios, eventos, alertas, dashboard, estadísticas y análisis.
- Motor de detección basado en reglas para operaciones masivas, alta frecuencia, fallos repetidos, horario inusual y desviación histórica.
- Persistencia MongoDB de usuarios de plataforma, eventos y alertas.
- Autenticación JWT, hash de contraseñas mediante Argon2 y autorización por roles admin/analyst.
- Simulador como fuente de eventos durante el MVP; la integración con una fuente real se considera parte de la preparación para producción.

No forman parte de la arquitectura actual el bloqueo automático de operaciones, modelos avanzados de aprendizaje automático, notificaciones externas, alta disponibilidad ni una integración nativa con todos los motores NoSQL.

1.3. Definición, siglas y abreviaturas

Término	Definición
API	Interfaz HTTP expuesta por FastAPI para autenticación, eventos, alertas, estadísticas, análisis y gestión.
Argon2	Algoritmo utilizado por la librería de hash para proteger contraseñas.
JWT	JSON Web Token empleado para autenticar solicitudes del frontend.
MVP	Producto mínimo viable utilizado para demostrar el flujo principal del sistema.
NoSQL	Familia de bases de datos no relacionales; el proyecto utiliza MongoDB.
QA	Atributo de calidad medible o evaluable que condiciona decisiones de arquitectura.
REST	Estilo de comunicación utilizado por los endpoints HTTP del backend.



Término	Definición
SAD	Software Architecture Document / Documento de Arquitectura de Software.
SRS	Software Requirements Specification / Especificación de Requerimientos de Software.
TLS	Mecanismo de cifrado requerido para HTTPS en despliegues externos.
UTC-5	Zona horaria de negocio utilizada para el análisis de horario inusual y actividad diaria.

1.4. Organización del documento

La sección 2 prioriza los requerimientos que influyen en la arquitectura y documenta las restricciones del proyecto. La sección 3 presenta las vistas del modelo 4+1 y los diagramas correspondientes. La sección 4 define escenarios de calidad con estímulo, respuesta y criterio de aceptación. Como documentos relacionados se consideran el Documento de Visión v1.0, el Informe de Factibilidad v1.0, el SRS v1.0 y el repositorio del proyecto en su rama funcional.



2. OBJETIVOS Y RESTRICCIONES ARQUITECTÓNICAS

La arquitectura prioriza la trazabilidad de eventos y alertas, la separación de responsabilidades, la seguridad de acceso al panel y la facilidad de despliegue del MVP. Las decisiones se orientan a mantener una solución comprensible para un equipo de dos integrantes y suficientemente modular para evolucionar hacia una integración real.

2.1. Priorización de requerimientos

La prioridad se deriva del SRS. Los requisitos de prioridad Alta condicionan el núcleo arquitectónico y deben conservarse durante cambios de implementación. Los de prioridad Media complementan la experiencia o la capacidad de crecimiento sin impedir el flujo principal.

2.1.1. Requerimientos Funcionales

ID	Descripción	Prioridad
RF-001	Gestionar inicio de sesión	Alta
RF-002	Autenticar y autorizar mediante JWT	Alta
RF-003	Gestionar roles Administrador y Analista	Alta
RF-004	Cerrar sesión	Alta
RF-005	Recibir eventos y validar su estructura	Alta
RF-006	Registrar eventos en MongoDB	Alta
RF-007	Analizar automáticamente los eventos	Alta
RF-008	Detectar operaciones masivas	Alta
RF-009	Detectar alta frecuencia	Alta
RF-010	Detectar fallos repetidos	Alta
RF-011	Detectar horario inusual	Alta
RF-012	Detectar desviación de comportamiento	Alta
RF-013	Calcular score y severidad	Alta
RF-014	Generar o agrupar alertas	Alta



ID	Descripción	Prioridad
RF-015	Consultar alertas y detalle	Alta
RF-016	Gestionar estado de alertas	Alta
RF-017	Visualizar dashboard	Alta
RF-018	Consultar y filtrar eventos	Alta
RF-019	Seleccionar usuario y fecha para análisis	Alta
RF-020	Resumir actividad diaria	Alta
RF-021	Visualizar actividad por hora e historial	Media
RF-022	Comparar comportamiento histórico	Alta
RF-023	Identificar cambios y filtrar análisis	Alta
RF-024	Gestionar cuentas de analistas	Alta
RF-025	Administrar acceso de analistas	Alta

2.1.2. Requerimientos No Funcionales - Atributos de Calidad

ID	Categoría	Descripción	Prioridad
RNF-001	Seguridad	Contraseñas protegidas mediante hash Argon2.	Alta
RNF-002	Seguridad	Acceso interno protegido con JWT y autorización por roles.	Alta



ID	Categoría	Descripción	Prioridad
RNF-003	Seguridad	La recepción de eventos externos debe usar credencial de servicio/API Key cuando se habilite una fuente autorizada.	Alta
RNF-004	Seguridad	Secretos y credenciales en variables de entorno, fuera del repositorio.	Alta
RNF-005	Seguridad	HTTPS/TLS obligatorio para despliegue externo.	Alta
RNF-006	Rendimiento	Consultas habituales con objetivo inferior a 3 s en condiciones normales del MVP.	Alta
RNF-007	Rendimiento	Análisis automático después de recibir un evento.	Alta
RNF-008	Integridad	Validación de entradas antes del almacenamiento.	Alta
RNF-009	Confiabilidad	Trazabilidad entre eventos, análisis y alertas.	Alta
RNF-010	Usabilidad	Indicadores, tablas, gráficos y mensajes comprensibles.	Alta
RNF-011	Compatibilidad	Funcionamiento en Chrome, Edge y Firefox modernos.	Media
RNF-012	Adaptabilidad	Interfaz responsive para escritorio, laptop, tablet y móvil.	Media



ID	Categoría	Descripción	Prioridad
RNF-013	Mantenibilidad	Separación de auth, datos, detección, servicios, rutas y frontend.	Alta
RNF-014	Escalabilidad	Crecimiento de eventos, usuarios y alertas sin reestructuración completa.	Media
RNF-015	Portabilidad	Instalación en entornos compatibles con Python/FastAPI/MongoDB mediante configuración externa.	Media

2.2. Restricciones

Restricción / decisión	Aplicación en el proyecto	Implicancia arquitectónica
Tecnología base	Backend Python/FastAPI y persistencia MongoDB ya forman parte del MVP; sustituirlos implicaría cambios de arquitectura y pruebas.	Favorece rapidez y coherencia con el curso, a costa de depender del ecosistema Python/MongoDB.
Frontend	HTML, CSS y JavaScript sin framework SPA; los archivos se sirven desde FastAPI.	Reduce complejidad de build y despliegue, pero limita patrones de frontend avanzados.
Fuente de eventos	El MVP usa simulator.py. La fuente productiva aún debe integrarse.	Permite pruebas controladas, pero no representa por sí sola una integración real.
Detección	Reglas explicables y estadística simple; umbrales en código/configuración.	Facilita auditoría y explicación, pero requiere calibración para reducir falsos positivos/negativos.



Restricción / decisión	Aplicación en el proyecto	Implicancia arquitectónica
Persistencia	Colecciones MongoDB users, events y alerts.	Modelo flexible para eventos semiestructurados; requiere índices, backups y políticas de retención en producción.
Seguridad de ingesta	POST /events está implementado sin una dependencia de autenticación de servicio en el código actual.	Antes de exponer la API se debe incorporar API Key, mTLS o restricción de red conforme al RNF-003.
Disponibilidad	No existe alta disponibilidad ni SLA en el MVP.	Adecuado para evaluación académica; un entorno crítico requiere redundancia y monitoreo.
Equipo	Dos integrantes participan en análisis, implementación, pruebas y documentación.	La arquitectura evita microservicios innecesarios y favorece un monolito modular mantenible.

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

La arquitectura se representa mediante las vistas definidas por la plantilla SAD. Los diagramas se elaboran con base en la estructura del repositorio y los requerimientos del SRS, manteniendo una separación entre el estado actual del MVP y los controles recomendados para producción.

3.1. Vista de Caso de uso

Los casos de uso arquitectónicamente significativos son aquellos que atraviesan varios componentes o imponen restricciones de seguridad y persistencia. El Administrador y el Analista utilizan el panel autenticado; la Fuente de eventos envía operaciones a la API; el sistema ejecuta la detección como parte del procesamiento del evento.

Caso de uso	Actor principal	Cobertura arquitectónica
Gestionar sesión	Administrador / Analista	Frontend, routes/auth.py, security.py, colección users y JWT.
Enviar y analizar evento	Fuente / Simulador	EventCreate, routes/events.py, MongoDB, engine.py, rules.py y alert_service.py.
Consultar / gestionar alertas	Administrador / Analista	routes/alerts.py, colección alerts y frontend de alertas.
Analizar actividad	Administrador / Analista	routes/analysis.py, consultas históricas, zona horaria y frontend Chart.js.
Gestionar analistas	Administrador	require_roles("admin"), routes/users.py y colección users.
Consultar dashboard / eventos	Administrador / Analista	dashboard.py, statistics.py, events.py y consultas MongoDB.

3.1.1. Diagramas de Casos de uso

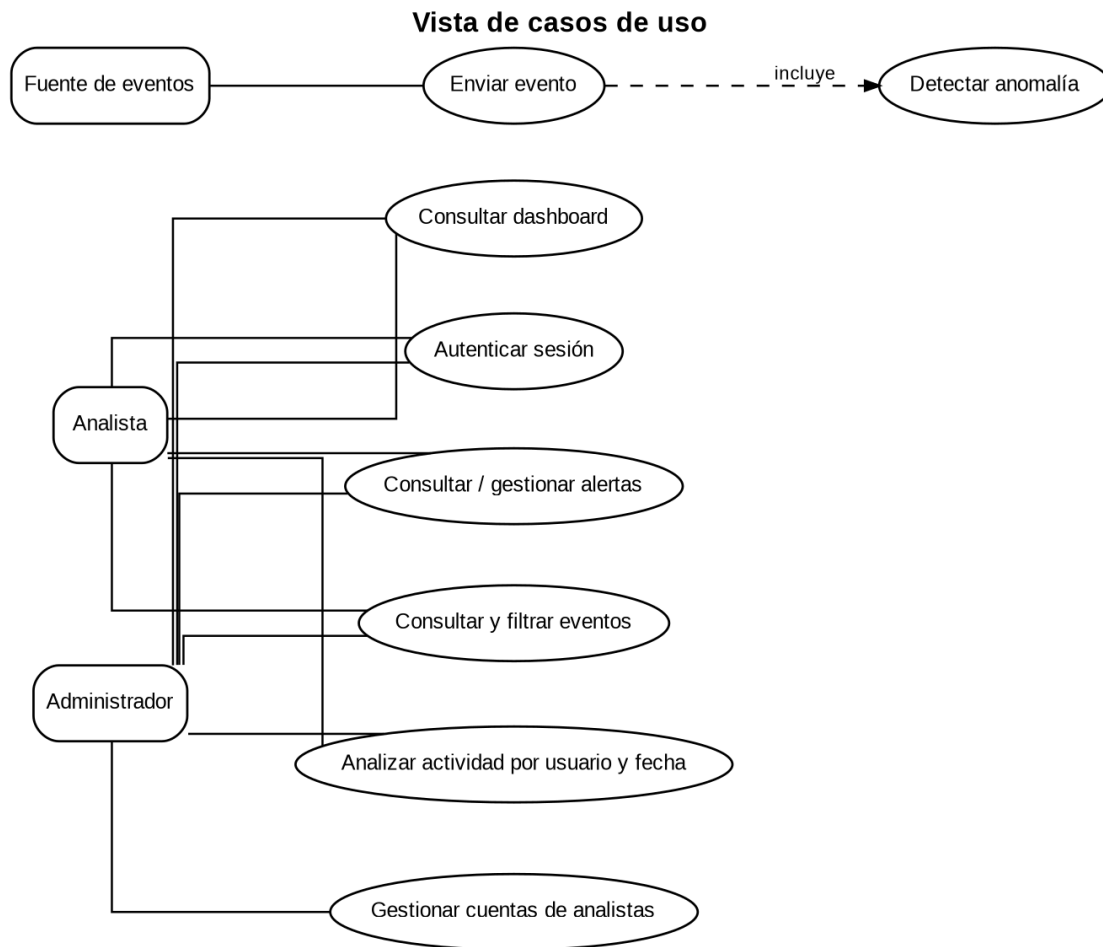


Figura 1. Casos de uso principales y actores del sistema.

3.2. Vista Lógica

La vista lógica organiza el sistema en subsistemas con responsabilidades separadas. El frontend no accede directamente a MongoDB; consume la API. Las rutas coordinan validación, autenticación, detección y persistencia. La lógica de detección se concentra en el paquete detection y la creación/agrupamiento de alertas se encapsula en services/alert_service.py.

3.2.1. Diagrama de Subsistemas (paquetes)

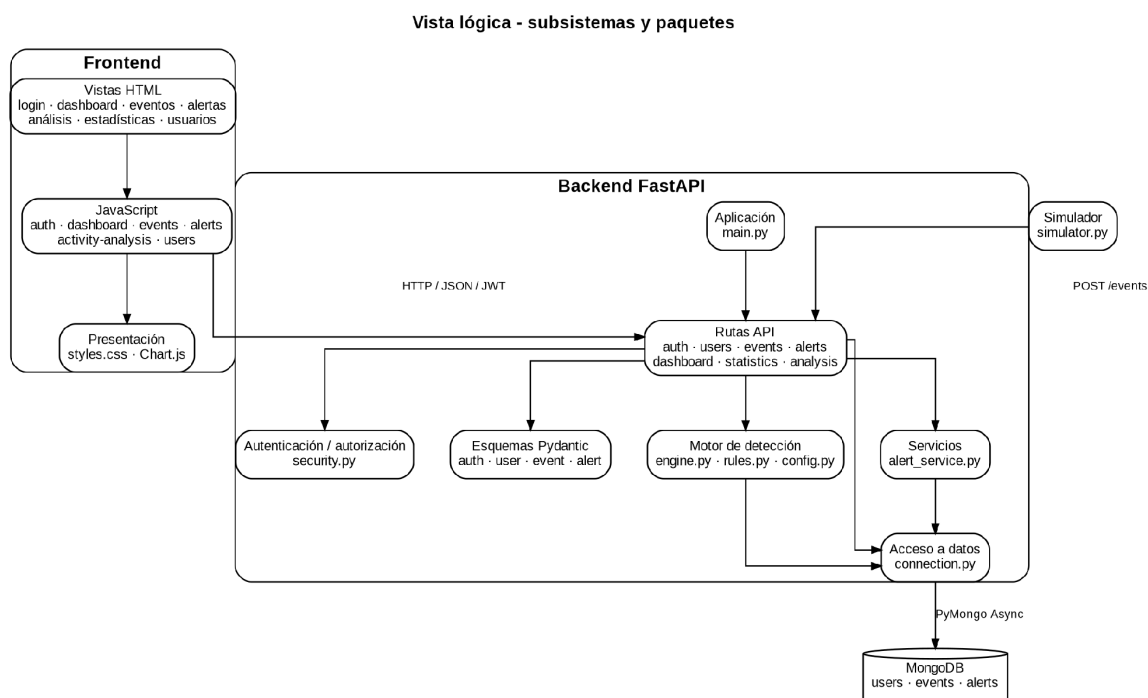


Figura 2. Subsistemas y paquetes principales del sistema.

Subsistema	Responsabilidad	Dependencias principales
Frontend	Interfaz, navegación, tablas y gráficos.	API FastAPI; Chart.js para visualización.
Rutas API	Exponer endpoints y coordinar casos de uso.	Schemas, auth, detection, services, database.
Autenticación	Hash de contraseñas, JWT, usuario actual y roles.	settings.py y colección users.
Detección	Aplicar reglas, sumar score y calcular severidad.	detection/config.py y consultas a events.
Servicios	Crear o agrupar alertas equivalentes.	Colección alerts.



Subsistema	Responsabilidad	Dependencias principales
Acceso a datos	Inicializar AsyncMongoClient y exponer database.	Variables de entorno y MongoDB.
Simulador	Generar eventos controlados para pruebas.	API POST /events.

3.2.2. Diagrama de Secuencia (vista de diseño)

El flujo de evento se realiza de forma asíncrona. El evento se inserta inicialmente en events, luego se analiza, se genera o agrupa una alerta si corresponde y finalmente se actualiza el mismo evento con analyzed, is_anomalous, anomaly_score, severity y alert_id.

Secuencia de recepción y análisis de un evento

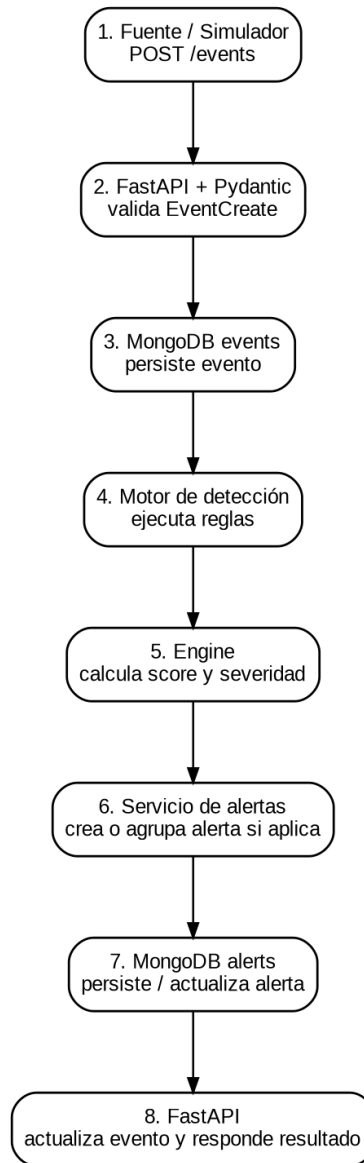


Figura 3. Secuencia de recepción, detección y persistencia.

La separación entre persistencia inicial y actualización posterior permite conservar el evento recibido y añadir el resultado del análisis. En una futura fase productiva conviene evaluar una estrategia transaccional o de recuperación para evitar estados intermedios si ocurre una falla entre ambas escrituras.



3.2.3. Diagrama de Colaboración (vista de diseño)

La colaboración evidencia que routes/events.py actúa como coordinador del caso de uso de ingesta. Pydantic valida el contrato; engine.py orquesta reglas; alert_service.py concentra la lógica de agrupamiento; connection.py centraliza el acceso a MongoDB.

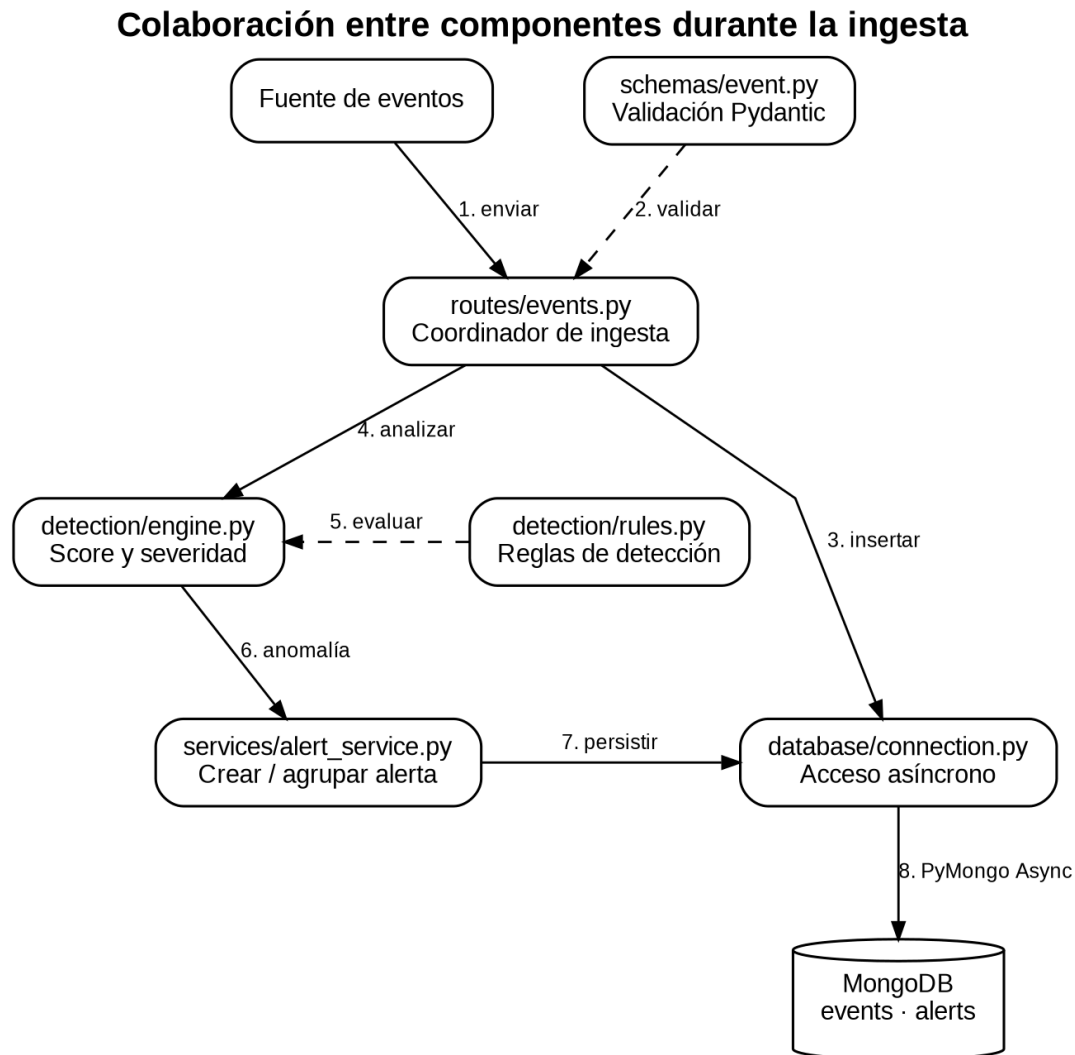


Figura 4. Colaboración entre componentes durante el procesamiento de un evento.

3.2.4. Diagrama de Objetos

El siguiente ejemplo ilustra una instantánea posible del sistema. Los valores son ilustrativos y sirven únicamente para mostrar la relación entre una identidad monitoreada, un evento detectado y la alerta generada.

Diagrama de objetos - ejemplo de una detección

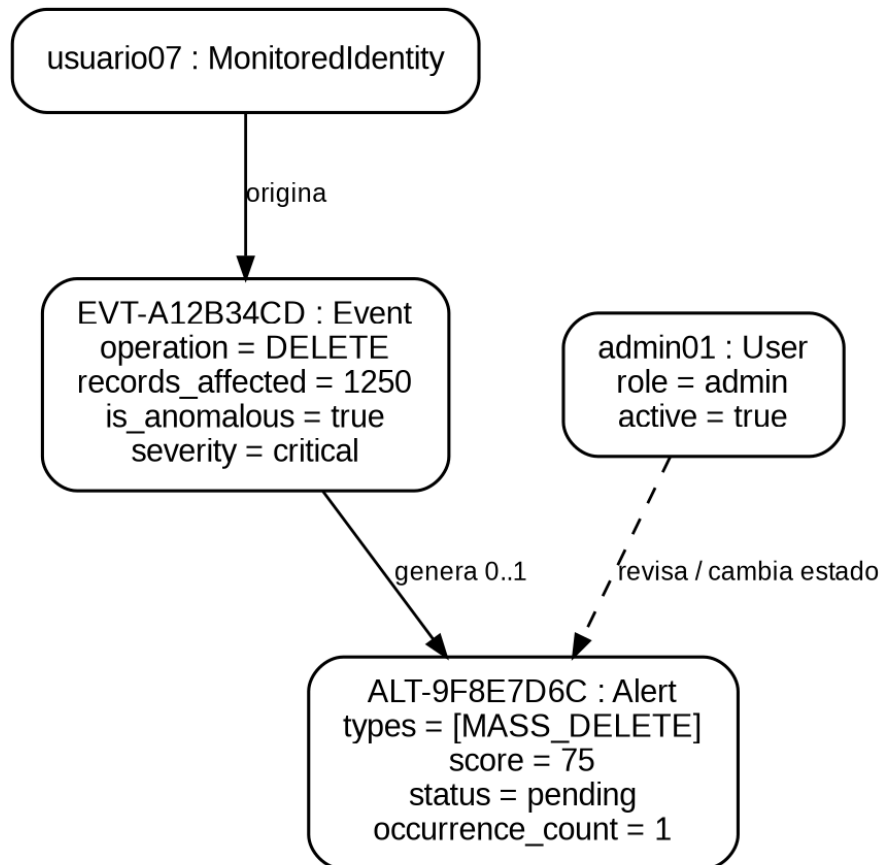


Figura 5. Ejemplo de objetos relacionados durante una detección.

3.2.5. Diagrama de Clases

El modelo de clases representa las entidades persistentes principales. Event y Alert se relacionan mediante event_id/alert_id; User corresponde a las cuentas de acceso del panel y no debe confundirse con user_id, que identifica al actor monitoreado dentro de un evento.

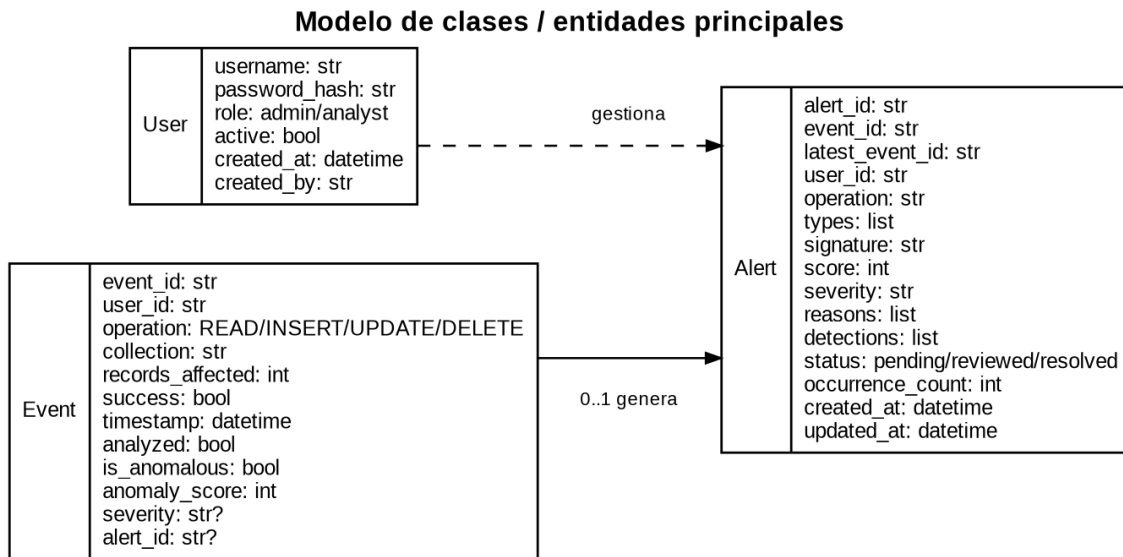


Figura 6. Entidades lógicas principales y relaciones.

3.2.6. Diagrama de Base de datos (NoSQL)

La base anomaly_detector utiliza tres colecciones principales. Los índices creados al iniciar la aplicación incluyen user_id + timestamp en events, username único en users y user_id + created_at en alerts. Estos índices responden a las consultas frecuentes del dashboard, filtros y análisis histórico.

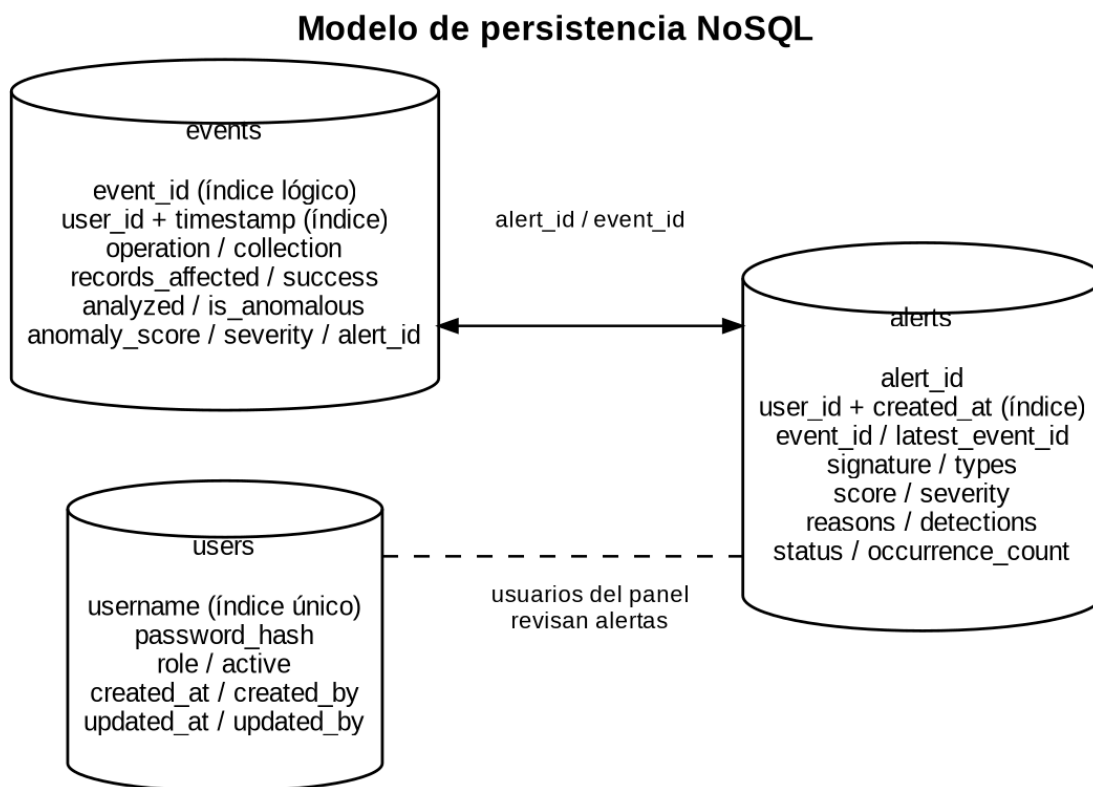


Figura 7. Modelo de persistencia NoSQL e índices principales.

Colección	Contenido	Índices / claves relevantes
events	Operación recibida, resultado de detección y referencia opcional a alerta.	event_id lógico; índice compuesto user_id + timestamp.
alerts	Detecciones agrupadas, score, severidad, razones, estado y ocurrencias.	alert_id lógico; índice compuesto user_id + created_at; signature para agrupamiento.
users	Cuentas de Administrador y Analista.	username único; password_hash; role; active.



3.3. Vista de Implementación (vista de desarrollo)

La vista de desarrollo mapea las responsabilidades lógicas a la estructura del repositorio. El backend está organizado bajo backend/app; el frontend se mantiene como archivos estáticos; el simulador y scripts de inicialización se encuentran separados del proceso principal.

3.3.1. Diagrama de arquitectura software (paquetes)

Vista de implementación - arquitectura software por paquetes

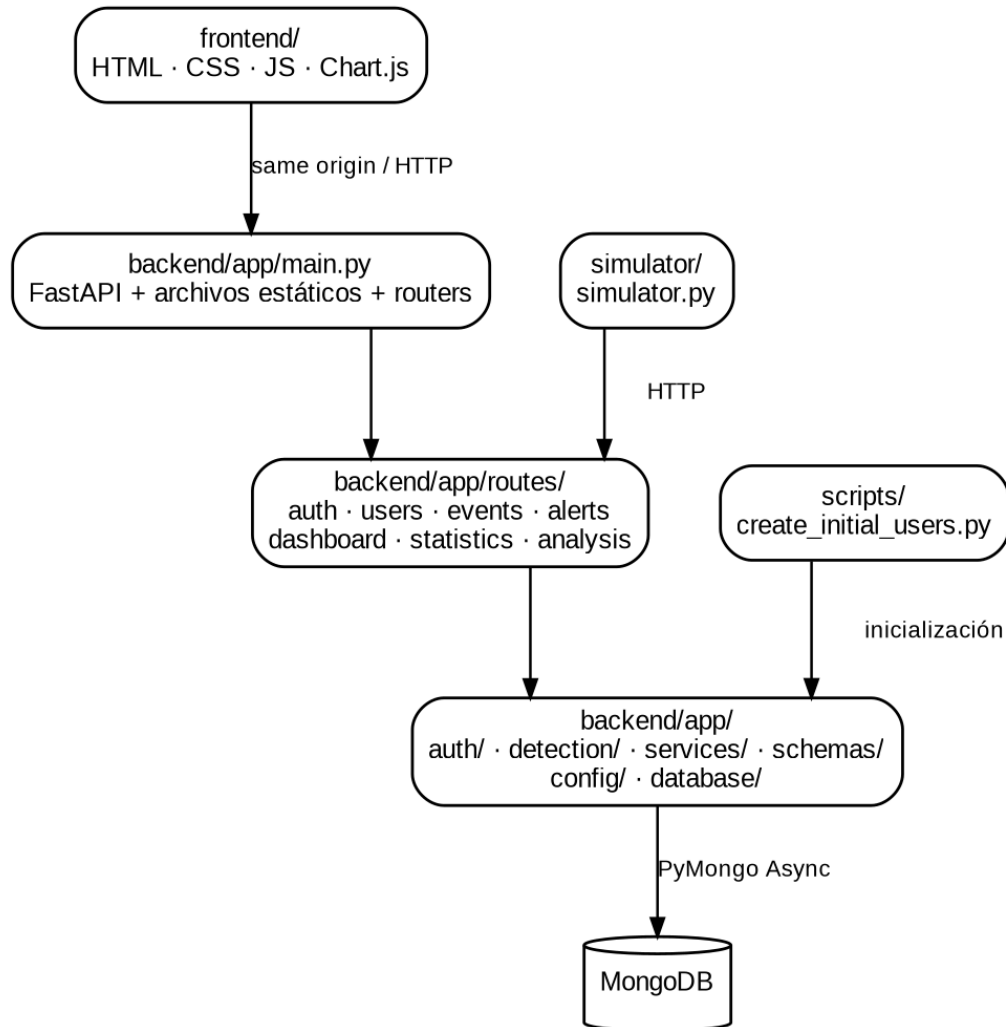


Figura 8. Arquitectura software según la organización del repositorio.

Ruta / paquete	Elementos relevantes	Función
backend/app/main.py	FastAPI, routers, StaticFiles, FileResponse, lifespan.	Punto de entrada, health checks, páginas y creación de índices.



Ruta / paquete	Elementos relevantes	Función
backend/app/routes	auth, users, events, alerts, dashboard, statistics, analysis.	Contratos HTTP de los casos de uso.
backend/app/auth	security.py.	Autenticación, JWT, hash y control de roles.
backend/app/detection	config.py, rules.py, engine.py.	Parámetros, reglas y orquestación de detección.
backend/app/services	alert_service.py.	Creación y agrupamiento temporal de alertas.
backend/app/schemas	auth.py, user.py, event.py, alert.py.	Validación de entrada/salida con Pydantic.
backend/app/database	connection.py.	AsyncMongoClient y acceso a la base configurada.
frontend	HTML, CSS y JavaScript.	Interfaz del usuario y consumo de endpoints.
simulator	simulator.py.	Generación de escenarios de prueba.
scripts	create_initial_users.py.	Inicialización de cuentas para el entorno de prueba.

3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)

La solución corresponde a un cliente web y un backend monolítico modular. La separación modular evita distribuir prematuramente la aplicación en microservicios, decisión coherente con el volumen del MVP, el tamaño del equipo y la necesidad de simplificar despliegue y depuración.

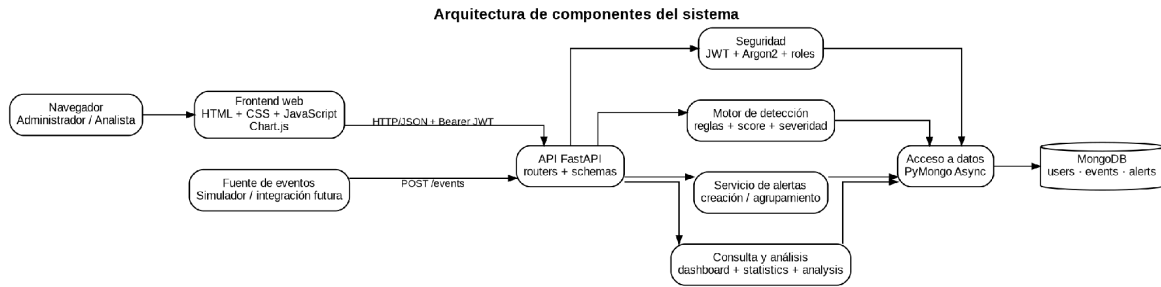


Figura 9. Componentes de ejecución y dependencias principales.

Interfaz	Emisor	Receptor	Formato / seguridad
Login	Frontend	POST /auth/login	Form URL encoded; respuesta JWT Bearer.
Sesión	Frontend	GET /auth/me	Bearer JWT.
Eventos	Fuente / Frontend	POST/GET /events	JSON; GET protegido con JWT. POST requiere endurecimiento para fuente externa.
Alertas	Frontend	GET/PATCH /alerts	JSON + Bearer JWT.
Dashboard	Frontend	GET /dashboard/summary	JSON + Bearer JWT.
Estadísticas	Frontend	GET /statistics/*	JSON + Bearer JWT.
Análisis	Frontend	GET /analysis/*	JSON + Bearer JWT.
Usuarios	Frontend admin	/users	JSON + Bearer JWT + rol admin.

3.4. Vista de procesos

El procesamiento principal se inicia con la recepción de un evento y finaliza con la disponibilidad del resultado para las vistas de monitoreo. Las operaciones de acceso a MongoDB utilizan AsyncMongoClient, por lo que las rutas pueden esperar operaciones de base de datos sin bloquear el servidor de la misma forma que una llamada síncrona tradicional.

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)

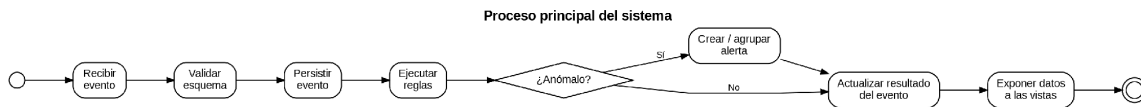


Figura 10. Actividad principal del sistema para el procesamiento de eventos.

Cuando el evento activa varias reglas, engine.py acumula los puntajes y limita el total a 100. La severidad se calcula como low para 1-24, medium para 25-49, high para 50-74 y critical para 75-100. Alertas pendientes con el mismo usuario y firma de detecciones pueden agruparse dentro de cinco minutos.

3.5. Vista de Despliegue (vista física)

Durante desarrollo, FastAPI y MongoDB pueden ejecutarse localmente. Para un piloto externo se recomienda separar el servidor de aplicación y la base administrada. La aplicación sirve el frontend y la API desde el mismo proceso, lo que simplifica el despliegue. El acceso externo debe realizarse mediante HTTPS y secretos almacenados fuera del repositorio.

3.5.1. Diagrama de despliegue

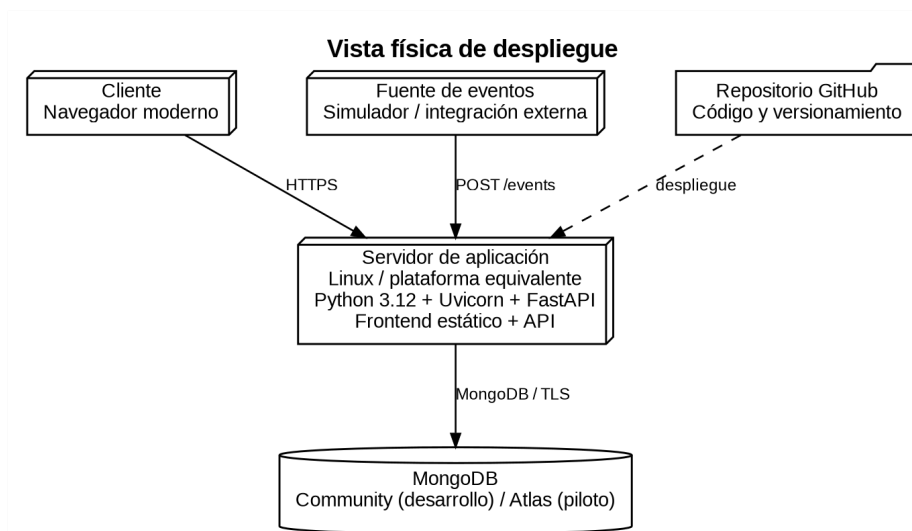


Figura 11. Vista física recomendada para desarrollo y piloto.

Nodo	Contenido / responsabilidad	Consideraciones
Cliente	Navegador de Administrador o Analista.	HTTPS, JavaScript habilitado y sesión JWT.
Servidor de aplicación	Python 3.12, Uvicorn/FastAPI, backend y frontend estático.	Variables de entorno, health checks, logs y reinicio controlado.
MongoDB	Community en desarrollo o Atlas para piloto.	Usuario de mínimo privilegio, TLS, backups, índices y monitoreo.
Fuente de eventos	Simulador actual o integración externa futura.	Autenticación de servicio / control de red antes de producción.
GitHub	Código fuente y control de versiones.	No incluir .env ni entornos virtuales en el artefacto de despliegue.



El repositorio ya excluye .env y .venv mediante .gitignore; sin embargo, el paquete de despliegue debe verificarse antes de publicarse para asegurar que no contenga credenciales, archivos locales o dependencias innecesarias.

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

Los atributos de calidad se expresan mediante escenarios concretos. Cada escenario identifica una fuente, un estímulo, el artefacto afectado, el entorno, la respuesta esperada y una medida de aceptación. Los valores se alinean con los RNF del SRS y con las capacidades actuales del MVP.

4.1. Escenario de Funcionalidad

Elemento del escenario	Definición para el sistema
Fuente	Administrador, Analista o fuente de eventos autorizada.
Estímulo	Ejecutar un flujo funcional del catálogo: autenticación, evento, alerta, consulta, análisis o gestión de usuarios.
Artefacto	Frontend, rutas FastAPI, motor de detección y MongoDB.
Entorno	MVP con servicios disponibles y datos de prueba precargados.
Respuesta	El sistema completa el flujo y conserva la trazabilidad entre entrada, procesamiento y persistencia.
Medida de aceptación	Los 25 RF pueden demostrarse según su alcance; las funciones críticas no presentan bloqueos que impidan el flujo principal.

4.2. Escenario de Usabilidad

Elemento del escenario	Definición para el sistema
Fuente	Administrador o Analista.
Estímulo	Ingresar al panel, filtrar eventos, revisar una alerta y ejecutar el análisis de un usuario.
Artefacto	Vistas HTML, JavaScript, navegación y gráficos Chart.js.
Entorno	Navegador moderno en resolución de escritorio o dispositivo compatible.
Respuesta	La interfaz muestra navegación consistente, estados, filtros y mensajes comprensibles.



Elemento del escenario	Definición para el sistema
Medida de aceptación	El usuario completa login, consulta y análisis sin instrucciones técnicas extensas; los errores no exponen secretos ni información interna sensible.

4.3. Escenario de Confiabilidad

Elemento del escenario	Definición para el sistema
Fuente	Aplicación o fuente de eventos.
Estímulo	Registrar eventos durante operación normal o consultar información persistida.
Artefacto	FastAPI, AsyncMongoClient y colecciones events/alerts/users.
Entorno	MongoDB disponible y configuración válida.
Respuesta	Cada evento recibe event_id, conserva timestamp y resultado de análisis; las alertas mantienen identificadores y estado.
Medida de aceptación	Sin pérdida de registros en operación normal; /health y /health/db permiten comprobar aplicación y base de datos. Ante indisponibilidad de MongoDB, el servicio no debe confirmar escrituras no realizadas.

4.4. Escenario de Rendimiento

Elemento del escenario	Definición para el sistema
Fuente	Administrador, Analista o simulador.
Estímulo	Solicitar dashboard, eventos, alertas, análisis o enviar un evento.
Artefacto	API FastAPI, consultas MongoDB y motor de detección.
Entorno	Carga de referencia del MVP/piloto de bajo volumen.

Elemento del escenario	Definición para el sistema
Respuesta	La API procesa la solicitud sin espera excesiva y aprovecha acceso asíncrono a MongoDB.
Medida de aceptación	Vistas principales con objetivo < 3 s y procesamiento de un evento con objetivo < 2 s en el entorno de referencia. Estos valores deben verificarse mediante pruebas antes del despliegue.

4.5. Escenario de Mantenibilidad

Elemento del escenario	Definición para el sistema
Fuente	Integrante del equipo de desarrollo.
Estímulo	Modificar una regla, un endpoint, un esquema o una vista.
Artefacto	Paquetes routes, detection, services, schemas, auth, database y frontend.
Entorno	Repositorio versionado y entorno de desarrollo reproducible.
Respuesta	El cambio se concentra en el módulo responsable y conserva contratos con los demás componentes.
Medida de aceptación	Una modificación de umbral se realiza en detection/config.py; una regla en rules.py; autenticación en security.py; persistencia en connection.py; cambios de interfaz permanecen en frontend.

4.6. Otros escenarios

Seguridad

Elemento del escenario	Definición para el sistema
Fuente	Usuario no autenticado, Analista sin privilegios o fuente externa.



Elemento del escenario	Definición para el sistema
Estímulo	Intentar acceder a una ruta protegida, ejecutar una acción administrativa o enviar eventos desde una fuente no autorizada.
Artefacto	security.py, dependencias FastAPI, routes/users.py y endpoint /events.
Entorno	Aplicación expuesta en red.
Respuesta	JWT inválido produce 401; rol insuficiente produce 403; las contraseñas se mantienen con hash Argon2. Para ingesta externa se debe exigir credencial de servicio o control de red.
Medida de aceptación	No se entregan password_hash ni secretos; .env queda fuera del repositorio; HTTPS habilitado en producción; POST /events se protege antes de exposición pública.

Portabilidad y despliegue

Elemento del escenario	Definición para el sistema
Fuente	Equipo de desarrollo / responsable de despliegue.
Estímulo	Instalar la aplicación en una máquina Linux o plataforma compatible.
Artefacto	requirements.txt, configuración por entorno, FastAPI y MongoDB.
Entorno	Servidor con Python 3.12 y acceso a MongoDB.
Respuesta	Se instalan dependencias, se definen MONGO_URI, MONGO_DB, SECRET_KEY y demás variables, y se inicia Uvicorn/FastAPI.
Medida de aceptación	/health responde ok y /health/db confirma conexión; el frontend se sirve desde la misma aplicación y las rutas protegidas funcionan con credenciales válidas.



Observaciones para el despliegue próximo

Antes de entregar una URL pública al docente o utilizar una fuente de eventos externa, conviene completar una verificación de despliegue: proteger POST /events, usar SECRET_KEY no predecible, habilitar HTTPS, configurar MongoDB con privilegios mínimos, crear datos de demostración, validar todos los roles y ejecutar pruebas de los flujos principales. Estas acciones no cambian el estilo arquitectónico del MVP; endurecen su operación.